Data Summarization

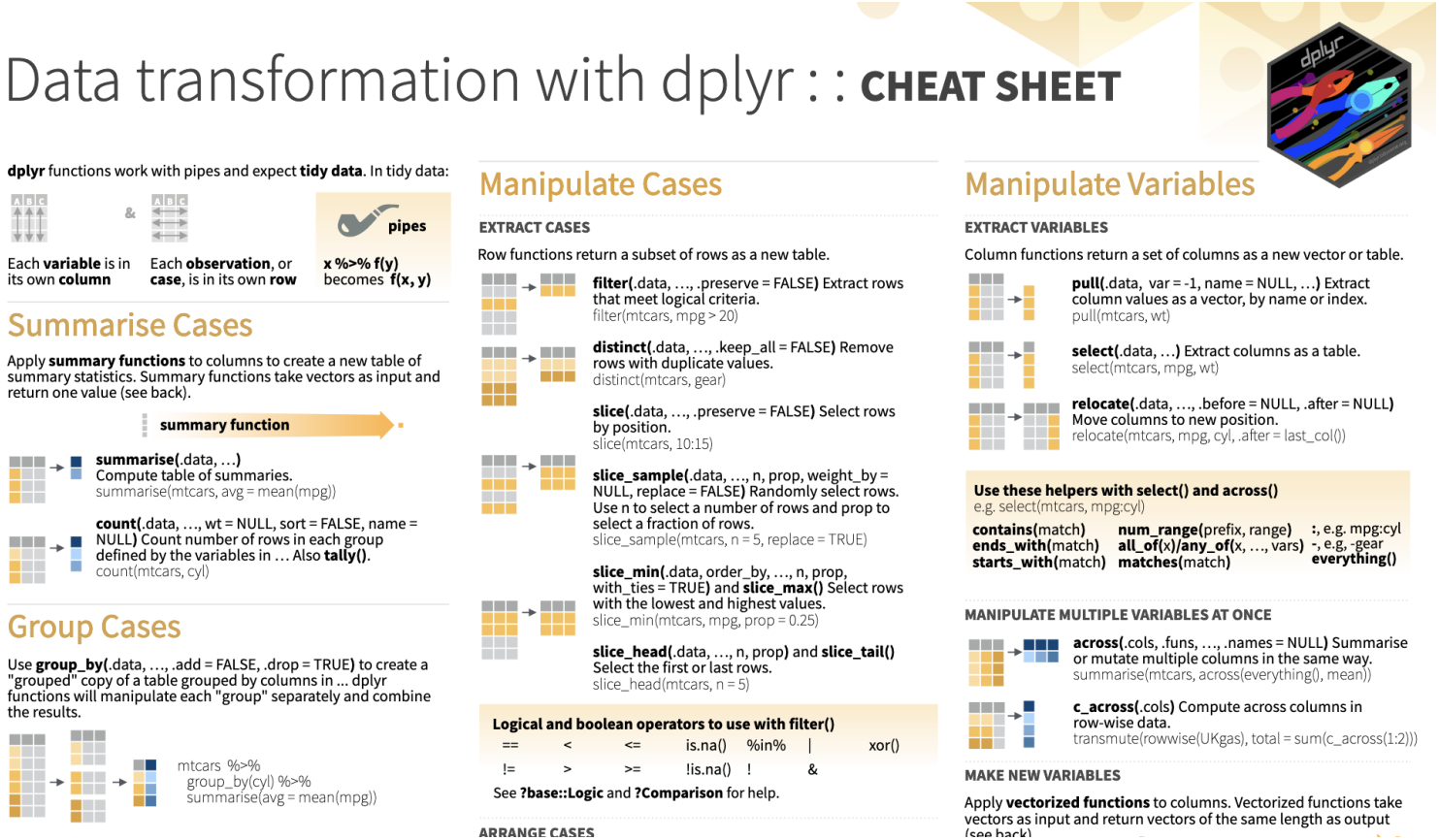
Recap

- `select()`: subset and/or reorder columns
- `filter()`: remove rows
- `arrange()`: reorder rows
- `mutate()`: create new columns or modify them
- `select()` and `filter()` can be combined together
- remove a column: `select()` with `!` mark (`!col_name`)
- you can do sequential steps: especially using pipes `%>%`

 Cheatsheet

Another Cheatsheet

https://raw.githubusercontent.com/rstudio/cheatsheets/main/data-transformation.pdf



Data transformation with dplyr : : CHEAT SHEET

dplyr functions work with pipes and expect **tidy data**. In tidy data:

Each **variable** is in its own **column**

&

Each **observation**, or **case**, is in its own **row**

pipes

x %>% f(y) becomes **f(x, y)**

Summarise Cases

Apply **summary functions** to columns to create a new table of summary statistics. Summary functions take vectors as input and return one value (see back).

summary function

summarise(.data, …**)**
Compute table of summaries.
summarise(mtcars, avg = mean(mpg))

count(.data, …, wt = NULL, sort = FALSE, name = NULL**)** Count number of rows in each group defined by the variables in … Also **tally()**.
count(mtcars, cyl)

Group Cases

Use **group_by(**.data, …, .add = FALSE, .drop = TRUE**)** to create a "grouped" copy of a table grouped by columns in ... dplyr functions will manipulate each "group" separately and combine the results.

```
mtcars %>%
  group_by(cyl) %>%
  summarise(avg = mean(mpg))
```

Manipulate Cases

EXTRACT CASES

Row functions return a subset of rows as a new table.

filter(.data, …, .preserve = FALSE**)** Extract rows that meet logical criteria.
filter(mtcars, mpg > 20)

distinct(.data, …, .keep_all = FALSE**)** Remove rows with duplicate values.
distinct(mtcars, gear)

slice(.data, …, .preserve = FALSE**)** Select rows by position.
slice(mtcars, 10:15)

slice_sample(.data, …, n, prop, weight_by = NULL, replace = FALSE**)** Randomly select rows. Use n to select a number of rows and prop to select a fraction of rows.
slice_sample(mtcars, n = 5, replace = TRUE)

slice_min(.data, order_by, …, n, prop, with_ties = TRUE**)** and **slice_max()** Select rows with the lowest and highest values.
slice_min(mtcars, mpg, prop = 0.25)

slice_head(.data, …, n, prop**)** and **slice_tail()** Select the first or last rows.
slice_head(mtcars, n = 5)

Logical and boolean operators to use with filter()

==	<	<=	is.na()	%in%	|	xor()
!=	>	>=	!is.na()	!	&	

See **?base::Logic** and **?Comparison** for help.

ARRANGE CASES

Manipulate Variables

EXTRACT VARIABLES

Column functions return a set of columns as a new vector or table.

pull(.data, var = -1, name = NULL, …**)** Extract column values as a vector, by name or index.
pull(mtcars, wt)

select(.data, …**)** Extract columns as a table.
select(mtcars, mpg, wt)

relocate(.data, …, .before = NULL, .after = NULL**)** Move columns to new position.
relocate(mtcars, mpg, cyl, .after = last_col())

Use these helpers with select() and across()
e.g. select(mtcars, mpg:cyl)

contains(match**)**
ends_with(match**)**
starts_with(match**)**
num_range(prefix, range**)**
all_of(x**)/any_of(**x, …, vars**)**
matches(match**)**
:, e.g. mpg:cyl
-, e.g, -gear
everything()

MANIPULATE MULTIPLE VARIABLES AT ONCE

across(.cols, .funs, …, .names = NULL**)** Summarise or mutate multiple columns in the same way.
summarise(mtcars, across(everything(), mean))

c_across(.cols**)** Compute across columns in row-wise data.
transmute(rowwise(UKgas), total = sum(c_across(1:2)))

MAKE NEW VARIABLES

Apply **vectorized functions** to columns. Vectorized functions take vectors as input and return vectors of the same length as output (see back).

Data Summarization

- Basic statistical summarization
  - `mean(x)`: takes the mean of x
  - `sd(x)`: takes the standard deviation of x
  - `median(x)`: takes the median of x
  - `quantile(x)`: displays sample quantiles of x. Default is min, IQR, max
  - `range(x)`: displays the range. Same as `c(min(x), max(x))`
  - `sum(x)`: sum of x
  - `max(x)`: maximum value in x
  - `min(x)`: minimum value in x
- **all have the `na.rm =` argument for missing data**

Statistical summarization

The vector getting summarized goes inside the parentheses:

```
x <- c(1, 5, 7, 4, 2, 8)
mean(x)
```

```
[1] 4.5
```

```
range(x)
```

```
[1] 1 8
```

```
sum(x)
```

```
[1] 27
```

Statistical summarization

Note that many of these functions have additional inputs regarding missing data, typically requiring the `na.rm` argument ("remove NAs").

```
x <- c(1, 5, 7, 4, 2, 8, NA)
mean(x)
```

```
[1] NA
```

```
mean(x, na.rm = TRUE)
```

```
[1] 4.5
```

```
quantile(x)
```

```
Error in quantile.default(x): missing values and NaN's not allowed if 'na.rm' is FALSE
```

```
quantile(x, na.rm = TRUE)
```

```
  0%  25%  50%  75% 100% 
 1.0  2.5  4.5  6.5  8.0 
```

Statistical summarization

We will talk more about data types later, but you can only do summarization on numeric or logical types. Not characters.

```
x <- c(1, 5, 7, 4, 2, 8)
sum(x)
```

```
[1] 27
```

```
y <- c(TRUE, FALSE, FALSE, TRUE) # FALSE == 0 and TRUE == 1
sum(y)
```

```
[1] 2
```

```
z <- c("TRUE", "FALSE", "FALSE", "TRUE")
sum(z)
```

```
Error in sum(z): invalid 'type' (character) of argument
```

Some examples

We can use the `mtcars` built-in dataset. “The data was extracted from the 1974 Motor Trend US magazine, and comprises fuel consumption and 10 aspects of automobile design and performance for 32 automobiles (1973-74 models).”

The `head` command displays the first rows of an object:

```
head(mtcars)
```

```
                  mpg cyl disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

The `dplyr` pipe `%>%` operator

A nice and readable way to chain together multiple R functions.

Changes `f(x, y)` to `x %>% f(y)`.

How mornings look like for most people:

```
me %>%
  wake_up() %>%
  get_out_of_bed() %>%
  get_dressed() %>%
  leave_house()
```

How my mornings look like most of the time:

```
leave_house(get_dressed(get_out_of_bed(wake_up(me))))
```

Statistical summarization the "tidy" way

```
mtcars %>% pull(hp) %>% mean() # alt: pull(mtcars, hp) %>% mean()
```

```
[1] 146.6875
```

```
mtcars %>% pull(wt) %>% median()
```

```
[1] 3.325
```

```
mtcars %>% pull(hp) %>% quantile()
```

```
   0%   25%   50%   75%  100% 
 52.0  96.5 123.0 180.0 335.0 
```

```
mtcars %>% pull(wt) %>% quantile(probs = 0.6)
```

```
 60% 
3.44 
```

Behavior of `pull()` function

`pull()` converts a single data column into a vector. This allows you to run summary functions on these data. Once you have "pulled" the data column out, you don't have to name it again in any piped summary functions.

```
cars_wt <- mtcars %>% pull(wt)
class(cars_wt)
```

```
[1] "numeric"
```

```
cars_wt
```

```
 [1] 2.620 2.875 2.320 3.215 3.440 3.460 3.570 3.190 3.150 3.440 3.440 4.070
[13] 3.730 3.780 5.250 5.424 5.345 2.200 1.615 1.835 2.465 3.520 3.435 3.840
[25] 3.845 1.935 2.140 1.513 3.170 2.770 3.570 2.780
```

```
mtcars %>% pull(wt) %>% range(wt) # Incorrect
```

```
mtcars %>% pull(wt) %>% range() # Correct
```

```
[1] 1.513 5.424
```

GUT CHECK

What kind of object do we need to run summary operators like `mean()` ?

A. A vector of numbers

B. A vector of characters

C. A dataset

Summarization on tibbles (data frames)

TB incidence

Let's read in a `tibble` of values from TB incidence.

"Tuberculosis incidence, all forms (per 100,000 population per year), for the period 1990-2007 across 208 countries/territories."

```
tb <- read_csv("https://jhudatascience.org/intro_to_r/data/tb.csv")
```

TB incidence

Check out the data:

```
head(tb)
```

```
# A tibble: 6 × 19
  TB incidence, all fo…¹ `1990` `1991` `1992` `1993` `1994` `1995` `1996` `1997`
  <chr>                   <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>  <dbl>
1 Afghanistan               168    168    168    168    168    168    168    168
2 Albania                    25     24     25     26     26     27     27     28
3 Algeria                    38     38     39     40     41     42     43     44
4 American Samoa             21      7      2      9      9     11      0     12
5 Andorra                    36     34     32     30     29     27     26     26
6 Angola                    205    209    214    218    222    226    231    236
# ℹ abbreviated name:
#   ¹`TB incidence, all forms (per 100 000 population per year)`
# ℹ 10 more variables: `1998` <dbl>, `1999` <dbl>, `2000` <dbl>, `2001` <dbl>,
#   `2002` <dbl>, `2003` <dbl>, `2004` <dbl>, `2005` <dbl>, `2006` <dbl>,
#   `2007` <dbl>
```

TB incidence

Check out the data:

```
str(tb)
```

```
spc_tbl_ [208 × 19] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ TB incidence, all forms (per 100 000 population per year): chr [1:208] "Afghanistan" "Albani
 $ 1990                                                     : num [1:208] 168 25 38 21 36 205 2
 $ 1991                                                     : num [1:208] 168 24 38 7 34 209 24
 $ 1992                                                     : num [1:208] 168 25 39 2 32 214 24
 $ 1993                                                     : num [1:208] 168 26 40 9 30 218 24
 $ 1994                                                     : num [1:208] 168 26 41 9 29 222 23
 $ 1995                                                     : num [1:208] 168 27 42 11 27 226 2
 $ 1996                                                     : num [1:208] 168 27 43 0 26 231 23
 $ 1997                                                     : num [1:208] 168 28 44 12 26 236 2
 $ 1998                                                     : num [1:208] 168 28 46 6 25 240 23
 $ 1999                                                     : num [1:208] 168 27 47 8 23 245 23
 $ 2000                                                     : num [1:208] 168 25 48 6 22 250 23
 $ 2001                                                     : num [1:208] 168 23 49 6 21 255 22
 $ 2002                                                     : num [1:208] 168 23 50 4 21 260 22
 $ 2003                                                     : num [1:208] 168 22 51 5 20 265 22
 $ 2004                                                     : num [1:208] 168 21 53 9 20 270 22
 $ 2005                                                     : num [1:208] 168 20 54 10 19 276 2
 $ 2006                                                     : num [1:208] 168 18 55 7 19 281 22
 $ 2007                                                     : num [1:208] 168 17 57 5 19 287 22
 - attr(*, "spec")=
  .. cols(
  ..   `TB incidence, all forms (per 100 000 population per year)` = col_character(),
  ..   `1990` = col_double(),
  ..   `1991` = col_double(),
```

Indicator of TB

Before we go further, let's rename the first column using the `rename()` function in `dplyr`.

In this case, we have to use the backticks (`) because there are spaces and funky characters in the name.

```
tb <- tb %>%
  rename(country = `TB incidence, all forms (per 100 000 population per year)`)
```

Indicator of TB

`colnames()` will show us the column names and show that country is renamed:

```
colnames(tb)
```

```
 [1] "country" "1990"    "1991"    "1992"    "1993"    "1994"    "1995"
 [8] "1996"    "1997"    "1998"    "1999"    "2000"    "2001"    "2002"
[15] "2003"    "2004"    "2005"    "2006"    "2007"
```

Summarize the data: `dplyr summarize()` function

`summarize` creates a summary table of a column you're interested in.

Can run multiple summary statistics at once (unlike `pull()` which can only do a single calculation on one column).

You can also do more elaborate summaries across different groups of data using `group_by()`. More on this later!

```
# General format - Not the code!
{data to use} %>%
   summarize({summary column name} = {function(source column)},
             {summary column name} = {function(source column)})
```

Summarize the data: `dplyr summarize()` function

`summarize` creates a summary table of a column you're interested in.

```
# General format - Not the code!
{data to use} %>%
   summarize({summary column name} = {function(source column)})

tb %>%
  summarize(mean_1991 = mean(`1991`)) # Note the backticks, this is a column name!

# A tibble: 1 × 1
  mean_1991
      <dbl>
1        NA

tb %>%
  summarize(mean_1991 = mean(`1991`, na.rm = TRUE))

# A tibble: 1 × 1
  mean_1991
      <dbl>
1      108.
```

Summarize the data: `dplyr summarize()` function

`summarize()` can do multiple operations at once. Just separate by a comma.

```
tb %>%
  summarize(mean_1991 = mean(`1991`, na.rm = TRUE),
            median_1991 = median(`1991`, na.rm = TRUE),
            median(`2000`, na.rm = TRUE))
```

```
# A tibble: 1 × 3
  mean_1991 median_1991 `median(\`2000\`, na.rm = TRUE)`
      <dbl>       <dbl>                            <dbl>
1      108.          58                               60
```

Notice how when we forget to provide a new name, output is still provided, but the column name is messy.

Summarize the data: `dplyr summarize()` function

This looks better.

```
tb %>%
  summarize(mean_1991 = mean(`1991`, na.rm = TRUE),
            median_1991 = median(`1991`, na.rm = TRUE),
            median_2000 = median(`2000`, na.rm = TRUE))

# A tibble: 1 × 3
  mean_1991 median_1991 median_2000
      <dbl>       <dbl>       <dbl>
1      108.          58          60
```

Summarize the data: `dplyr summarize()` function

Note that `summarize()` creates a separate tibble from the original data, so you don't want to overwrite your original data if you decide to save the summary.

If you want to save a summary statistic in the original data, use `mutate()` instead to create a new column for the summary statistic.

`summary()` Function

Using `summary()` can give you rough snapshots of each numeric column (character columns are skipped):

```
summary(tb)
```

```
  country              1990            1991            1992
Length:208         Min.   :  0.0   Min.   :  4.0   Min.   :  2.0
Class :character   1st Qu.: 27.5   1st Qu.: 27.0   1st Qu.: 27.0
Mode  :character   Median : 60.0   Median : 58.0   Median : 56.0
                   Mean   :105.6   Mean   :107.7   Mean   :108.3
                   3rd Qu.:165.0   3rd Qu.:171.0   3rd Qu.:171.5
                   Max.   :585.0   Max.   :594.0   Max.   :606.0
                   NA's   :1       NA's   :1       NA's   :1
     1993            1994            1995            1996            1997
Min.   :  4.0   Min.   :  0   Min.   :  3.0   Min.   :  0.0   Min.   :  0.0
1st Qu.: 27.5   1st Qu.: 26   1st Qu.: 26.5   1st Qu.: 25.5   1st Qu.: 24.5
Median : 56.0   Median : 57   Median : 58.0   Median : 60.0   Median : 64.0
Mean   :110.3   Mean   :112   Mean   :114.2   Mean   :115.4   Mean   :118.9
3rd Qu.:171.0   3rd Qu.:174   3rd Qu.:177.5   3rd Qu.:179.0   3rd Qu.:181.0
Max.   :618.0   Max.   :630   Max.   :642.0   Max.   :655.0   Max.   :668.0
NA's   :1       NA's   :1     NA's   :1       NA's   :1       NA's   :1
     1998            1999            2000            2001
Min.   :  0.0   Min.   :  0.0   Min.   :  0.0   Min.   :  0.0
1st Qu.: 23.5   1st Qu.: 22.5   1st Qu.: 21.5   1st Qu.: 19.0
Median : 63.0   Median : 66.0   Median : 60.0   Median : 59.0
Mean   :121.5   Mean   :125.0   Mean   :127.8   Mean   :130.7
3rd Qu.:188.5   3rd Qu.:192.5   3rd Qu.:191.0   3rd Qu.:189.5
Max.   :681.0   Max.   :695.0   Max.   :801.0   Max.   :916.0
NA's   :1       NA's   :1       NA's   :1       NA's   :1
```

Summary & Lab Part 1

- `pull()` creates a *vector*
- don't forget the `na.rm = TRUE` argument!
- `summary(x)`: quantile information
- `summarize`: creates a summary table of columns of interest
- summary stats (`mean()`) work with vectors or with `summarize()`

- Class Website
- Lab
- Day 4 Cheatsheet

Colorado heat-related ER visits

Here we will be using data on the annual rates of medical visits associated with heat-related illness in Colorado: http://jhudatascience.org/intro_to_r/data/CO_ER_heat_visits_2011_2024.csv

- Check out info about the data at: https://coepht.colorado.gov/heat-data

```
CO_heat <- read_csv("http://jhudatascience.org/intro_to_r/data/CO_ER_heat_visits_2011_2024.csv")
head(CO_heat)
```

```
# A tibble: 6 × 11
  COUNTY     RATE L95CL U95CL VISITS  FLAG  YEAR GENDER  MEASURE HEALTHOUTCOMEID
  <chr>     <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <chr>   <chr>   <chr>
1 Statewide  5.65  4.70  6.60    140     2  2011 Female  Age ad… Heat-related
2 Statewide  7.39  6.30  8.48    183     2  2011 Male    Age ad… Heat-related
3 Statewide  6.52  5.80  7.24    323     2  2011 Both g… Age ad… Heat-related
4 Statewide  5.65  4.72  6.58    146     2  2012 Female  Age ad… Heat-related
5 Statewide  7.57  6.48  8.65    193     2  2012 Male    Age ad… Heat-related
6 Statewide  6.59  5.88  7.30    339     2  2012 Both g… Age ad… Heat-related
# ℹ 1 more variable: cofips <chr>
```

`distinct()` values

`distinct(x)` will return the unique elements of column `x`.

```
CO_heat %>%
  distinct(COUNTY)
```

```
# A tibble: 65 × 1
   COUNTY
   <chr>
 1 Statewide
 2 Adams
 3 Alamosa
 4 Arapahoe
 5 Archuleta
 6 Baca
 7 Bent
 8 Boulder
 9 Broomfield
10 Chaffee
# ℹ 55 more rows
```

How many `distinct()` values?

`n_distinct()` tells you the number of unique elements. It needs a vector so you *must pull the column first!*

```
CO_heat %>%
  pull(COUNTY) %>%
  n_distinct()
```

```
[1] 65
```

Use **count( )** to return row count per category.

Use `count` to return a frequency table of unique elements of a data.frame.

```
CO_heat %>% count(COUNTY)
```

```
# A tibble: 65 × 2
   COUNTY         n
   <chr>      <int>
 1 Adams         42
 2 Alamosa       42
 3 Arapahoe      42
 4 Archuleta     42
 5 Baca          42
 6 Bent          42
 7 Boulder       42
 8 Broomfield    42
 9 Chaffee       42
10 Cheyenne      42
# ℹ 55 more rows
```

Multiple columns listed further subdivides the **count( )**

```
CO_heat %>% count(COUNTY, GENDER)
```

```
# A tibble: 195 × 3
   COUNTY    GENDER           n
   <chr>     <chr>        <int>
 1 Adams     Both genders    14
 2 Adams     Female          14
 3 Adams     Male            14
 4 Alamosa   Both genders    14
 5 Alamosa   Female          14
 6 Alamosa   Male            14
 7 Arapahoe  Both genders    14
 8 Arapahoe  Female          14
 9 Arapahoe  Male            14
10 Archuleta Both genders    14
# ℹ 185 more rows
```

Note: `count()` includes NAs

GUT CHECK

The `count()` function can help us tally:

A. Sample size

B. Rows per each category

C. How many categories

Grouping

Goal

We want to find the average rate for ER visits for heat-related illness in the dataset.

How do we do this?

Perform operations By groups: dplyr

`group_by` allows you group the data set by variables/columns you specify:

```
# Regular data
CO_heat
```

```
# A tibble: 2,730 × 11
   COUNTY     RATE L95CL U95CL VISITS  FLAG  YEAR GENDER MEASURE HEALTHOUTCOMEID
   <chr>     <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <chr>  <chr>   <chr>
 1 Statewide  5.65  4.70  6.60    140     2  2011 Female Age ad… Heat-related
 2 Statewide  7.39  6.30  8.48    183     2  2011 Male   Age ad… Heat-related
 3 Statewide  6.52  5.80  7.24    323     2  2011 Both … Age ad… Heat-related
 4 Statewide  5.65  4.72  6.58    146     2  2012 Female Age ad… Heat-related
 5 Statewide  7.57  6.48  8.65    193     2  2012 Male   Age ad… Heat-related
 6 Statewide  6.59  5.88  7.30    339     2  2012 Both … Age ad… Heat-related
 7 Statewide  4.96  4.08  5.84    124     2  2013 Female Age ad… Heat-related
 8 Statewide  6.70  5.70  7.70    178     2  2013 Male   Age ad… Heat-related
 9 Statewide  5.83  5.16  6.49    302     2  2013 Both … Age ad… Heat-related
10 Statewide  3.54  2.81  4.27     92     2  2014 Female Age ad… Heat-related
# ℹ 2,720 more rows
# ℹ 1 more variable: cofips <chr>
```

Perform operations by groups: dplyr

`group_by` allows you group the data set by variables/columns you specify:

```
CO_heat_grouped <- CO_heat %>% group_by(COUNTY)
CO_heat_grouped
```

```
# A tibble: 2,730 × 11
# Groups:   COUNTY [65]
   COUNTY     RATE L95CL U95CL VISITS  FLAG  YEAR GENDER MEASURE HEALTHOUTCOMEID
   <chr>     <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <chr>  <chr>   <chr>
 1 Statewide  5.65  4.70  6.60    140     2  2011 Female Age ad… Heat-related
 2 Statewide  7.39  6.30  8.48    183     2  2011 Male   Age ad… Heat-related
 3 Statewide  6.52  5.80  7.24    323     2  2011 Both … Age ad… Heat-related
 4 Statewide  5.65  4.72  6.58    146     2  2012 Female Age ad… Heat-related
 5 Statewide  7.57  6.48  8.65    193     2  2012 Male   Age ad… Heat-related
 6 Statewide  6.59  5.88  7.30    339     2  2012 Both … Age ad… Heat-related
 7 Statewide  4.96  4.08  5.84    124     2  2013 Female Age ad… Heat-related
 8 Statewide  6.70  5.70  7.70    178     2  2013 Male   Age ad… Heat-related
 9 Statewide  5.83  5.16  6.49    302     2  2013 Both … Age ad… Heat-related
10 Statewide  3.54  2.81  4.27     92     2  2014 Female Age ad… Heat-related
# ℹ 2,720 more rows
# ℹ 1 more variable: cofips <chr>
```

Summarize the grouped data

It's grouped! Grouping doesn't change the data in any way, but how **functions operate on it**. Now we can summarize RATE (age-adjusted heat-related ER visits per 100,000 people) by group:

```
CO_heat_grouped %>% summarize(avg_rate = mean(RATE, na.rm = TRUE))
```

```
# A tibble: 65 × 2
   COUNTY     avg_rate
   <chr>         <dbl>
 1 Adams          6.85
 2 Alamosa        0
 3 Arapahoe       5.20
 4 Archuleta      0
 5 Baca           0
 6 Bent           0
 7 Boulder        5.46
 8 Broomfield     0
 9 Chaffee        0
10 Cheyenne       0
# ℹ 55 more rows
```

Do it in one step: use %>% to string these together!

Pipe `CO_heat` into `group_by`, then pipe that into `summarize`:

```
CO_heat %>%
  group_by(COUNTY) %>%
  summarize(avg_rate = mean(RATE, na.rm = TRUE),
            max_rate = max(RATE, na.rm = TRUE))
```

```
# A tibble: 65 × 3
   COUNTY     avg_rate max_rate
   <chr>         <dbl>    <dbl>
 1 Adams          6.85    11.7
 2 Alamosa        0        0
 3 Arapahoe       5.20     8.44
 4 Archuleta      0        0
 5 Baca           0        0
 6 Bent           0        0
 7 Boulder        5.46     8.96
 8 Broomfield     0        0
 9 Chaffee        0        0
10 Cheyenne       0        0
# ℹ 55 more rows
```

Group by as many variables as you want

`group_by` COUNTY and GENDER:

```
CO_heat %>%
  group_by(COUNTY, GENDER) %>%
  summarize(avg_rate = mean(RATE, na.rm = TRUE),
            max_rate = max(RATE, na.rm = TRUE))
```

```
Warning: There were 4 warnings in `summarize()`.
The first warning was:
 In argument: `max_rate = max(RATE, na.rm = TRUE)`.
 In group 20: `COUNTY = "Boulder"` `GENDER = "Female"`.
Caused by warning in `max()`:
! no non-missing arguments to max; returning -Inf
 Run `dplyr::last_dplyr_warnings()` to see the 3 remaining warnings.
```

```
# A tibble: 195 × 4
# Groups:   COUNTY [65]
   COUNTY    GENDER       avg_rate max_rate
   <chr>     <chr>           <dbl>    <dbl>
 1 Adams     Both genders     6.35     8.96
 2 Adams     Female           6.37     7.59
 3 Adams     Male             7.83    11.7
 4 Alamosa   Both genders     0        0
 5 Alamosa   Female           0        0
 6 Alamosa   Male             0        0
 7 Arapahoe  Both genders     4.93     7.16
 8 Arapahoe  Female           4.77     5.86
 9 Arapahoe  Male             5.94     8.44
```

Only the last `group_by` is recognized…

You can overwrite the first `group_by` with a new one.

```
CO_heat %>%
  group_by(COUNTY, GENDER)  %>%
  group_by(GENDER)
```

```
# A tibble: 2,730 × 11
# Groups:   GENDER [3]
   COUNTY     RATE L95CL U95CL VISITS  FLAG  YEAR GENDER MEASURE HEALTHOUTCOMEID
   <chr>     <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <chr>  <chr>   <chr>
 1 Statewide  5.65  4.70  6.60    140     2  2011 Female Age ad… Heat-related
 2 Statewide  7.39  6.30  8.48    183     2  2011 Male   Age ad… Heat-related
 3 Statewide  6.52  5.80  7.24    323     2  2011 Both … Age ad… Heat-related
 4 Statewide  5.65  4.72  6.58    146     2  2012 Female Age ad… Heat-related
 5 Statewide  7.57  6.48  8.65    193     2  2012 Male   Age ad… Heat-related
 6 Statewide  6.59  5.88  7.30    339     2  2012 Both … Age ad… Heat-related
 7 Statewide  4.96  4.08  5.84    124     2  2013 Female Age ad… Heat-related
 8 Statewide  6.70  5.70  7.70    178     2  2013 Male   Age ad… Heat-related
 9 Statewide  5.83  5.16  6.49    302     2  2013 Both … Age ad… Heat-related
10 Statewide  3.54  2.81  4.27     92     2  2014 Female Age ad… Heat-related
# ℹ 2,720 more rows
# ℹ 1 more variable: cofips <chr>
```

Ungroup the data

The `ungroup` function will allow you to clear the groups from the data.

```
CO_heat_grouped
```

```
# A tibble: 2,730 × 11
# Groups:   COUNTY [65]
   COUNTY     RATE L95CL U95CL VISITS  FLAG  YEAR GENDER MEASURE HEALTHOUTCOMEID
   <chr>     <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <chr>  <chr>   <chr>
 1 Statewide  5.65  4.70  6.60    140     2  2011 Female Age ad… Heat-related
 2 Statewide  7.39  6.30  8.48    183     2  2011 Male   Age ad… Heat-related
 3 Statewide  6.52  5.80  7.24    323     2  2011 Both … Age ad… Heat-related
 4 Statewide  5.65  4.72  6.58    146     2  2012 Female Age ad… Heat-related
 5 Statewide  7.57  6.48  8.65    193     2  2012 Male   Age ad… Heat-related
 6 Statewide  6.59  5.88  7.30    339     2  2012 Both … Age ad… Heat-related
 7 Statewide  4.96  4.08  5.84    124     2  2013 Female Age ad… Heat-related
 8 Statewide  6.70  5.70  7.70    178     2  2013 Male   Age ad… Heat-related
 9 Statewide  5.83  5.16  6.49    302     2  2013 Both … Age ad… Heat-related
10 Statewide  3.54  2.81  4.27     92     2  2014 Female Age ad… Heat-related
# ℹ 2,720 more rows
# ℹ 1 more variable: cofips <chr>
```

```
ungroup(CO_heat_grouped)
```

```
# A tibble: 2,730 × 11
   COUNTY     RATE L95CL U95CL VISITS  FLAG  YEAR GENDER MEASURE HEALTHOUTCOMEID
   <chr>     <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <chr>  <chr>   <chr>
 1 Statewide  5.65  4.70  6.60    140     2  2011 Female Age ad… Heat-related
 2 Statewide  7.39  6.30  8.48    183     2  2011 Male   Age ad… Heat-related
 3 Statewide  6.52  5.80  7.24    323     2  2011 Both … Age ad… Heat-related
```

group_by with mutate - just add data

We can also use `mutate` to calculate the mean value for each year and add it as a column:

```
CO_heat %>%
  group_by(YEAR, GENDER) %>%
  mutate(year_avg = mean(RATE, na.rm = TRUE)) %>%
  select(COUNTY, RATE, year_avg)
```

```
# A tibble: 2,730 × 5
# Groups:   YEAR, GENDER [42]
    YEAR GENDER       COUNTY     RATE year_avg
   <dbl> <chr>        <chr>     <dbl>    <dbl>
 1  2011 Female       Statewide  5.65    0.950
 2  2011 Male         Statewide  7.39    1.38
 3  2011 Both genders Statewide  6.52    1.59
 4  2012 Female       Statewide  5.65    0.891
 5  2012 Male         Statewide  7.57    1.71
 6  2012 Both genders Statewide  6.59    1.86
 7  2013 Female       Statewide  4.96    0.795
 8  2013 Male         Statewide  6.70    1.65
 9  2013 Both genders Statewide  5.83    1.92
10  2014 Female       Statewide  3.54    0.508
# ℹ 2,720 more rows
```

Counting

There are other functions, such as `n()` count the number of observations (NAs included).

```
CO_heat %>%
  group_by(YEAR) %>%
  summarize(n = n(),
            mean = mean(RATE, na.rm = TRUE))
```

```
# A tibble: 14 × 3
    YEAR     n  mean
   <dbl> <int> <dbl>
 1  2011   195 1.29
 2  2012   195 1.50
 3  2013   195 1.45
 4  2014   195 0.967
 5  2015   195 1.54
 6  2016   195 3.19
 7  2017   195 1.55
 8  2018   195 2.84
 9  2019   195 3.34
10  2020   195 1.26
11  2021   195 1.68
12  2022   195 2.45
13  2023   195 2.60
14  2024   195 2.62
```

Counting

`count()` and `n()` can give very similar information.

```
CO_heat %>% count(YEAR) %>% head(n = 3)
```

```
# A tibble: 3 × 2
   YEAR     n
  <dbl> <int>
1  2011   195
2  2012   195
3  2013   195
```

```
CO_heat %>% group_by(YEAR) %>% summarize(n = n()) %>% head(n = 3) # n() typically used with summarize
```

```
# A tibble: 3 × 2
   YEAR     n
  <dbl> <int>
1  2011   195
2  2012   195
3  2013   195
```

A few miscellaneous topics

Base R functions you might see: `length` and `unique`

These functions require a column as a vector using `pull()`.

```
CO_heat_loc <- CO_heat %>% pull(COUNTY) # pull() to make a vector
CO_heat_loc %>% unique() # similar to distinct()
```

```
 [1] "Statewide"   "Adams"       "Alamosa"     "Arapahoe"    "Archuleta"
 [6] "Baca"        "Bent"        "Boulder"     "Broomfield"  "Chaffee"
[11] "Cheyenne"    "Clear Creek" "Conejos"     "Costilla"    "Crowley"
[16] "Custer"      "Delta"       "Denver"      "Dolores"     "Douglas"
[21] "Eagle"       "Elbert"      "El Paso"     "Fremont"     "Garfield"
[26] "Gilpin"      "Grand"       "Gunnison"    "Hinsdale"    "Huerfano"
[31] "Jackson"     "Jefferson"   "Kiowa"       "Kit Carson"  "Lake"
[36] "La Plata"    "Larimer"     "Las Animas"  "Lincoln"     "Logan"
[41] "Mesa"        "Mineral"     "Moffat"      "Montezuma"   "Montrose"
[46] "Morgan"      "Otero"       "Ouray"       "Park"        "Phillips"
[51] "Pitkin"      "Prowers"     "Pueblo"      "Rio Blanco"  "Rio Grande"
[56] "Routt"       "Saguache"    "San Juan"    "San Miguel"  "Sedgwick"
[61] "Summit"      "Teller"      "Washington"  "Weld"        "Yuma"
```

Base R functions you might see: `length` and `unique`

These functions require a column as a vector using `pull()`.

```
CO_heat_loc %>% unique() %>% length() # similar to n_distinct()
```

```
[1] 65
```

`summary()` vs. `summarize()`

- `summary()` (base R) gives statistics table on a dataset.
- `summarize()` (dplyr) creates a more customized summary tibble/dataframe.

Functions you might also see

- `sum(!is.na())`: # of non-NAs in the data
- `first()`: first value in the data
- `last()`: last value in the data
- `range()`: minimum and maximum of the data
- `IQR()`: interquartile range of the data

Summary

- `count(x)`: what unique values do you have?
  - `distinct()`: what are the distinct values?
  - `n_distinct()` with `pull()`: how many distinct values?
- `group_by()`: changes subsequent functions (remove with `ungroup()`)
  - combine with `summarize()` to get statistics per group
  - combine with `mutate()` to add column
- `summarize()` with `n()` gives the count (NAs included)

Resources & Lab Part 2

- Class Website
- Lab
- Day 4 Cheatsheet
- Posit's data transformation Cheatsheet

Image by Gerd Altmann from Pixabay

Extra Slides: More advanced summarization

Data Summarization on data frames

- Statistical summarization across the data frame
  - `rowMeans(x)`: takes the means of each row of x
  - `colMeans(x)`: takes the means of each column of x
  - `rowSums(x)`: takes the sum of each row of x
  - `colSums(x)`: takes the sum of each column of x

`rowMeans()` example

Get means for each row.

Let's see what the mean TB incidence is across years each row (country):

```
tb %>%
  select(starts_with("year")) %>%
  rowMeans(na.rm = TRUE) %>%
  head(n = 5)
```

```
[1] NaN NaN NaN NaN NaN
```

```
tb %>%
  group_by(country) %>%
  summarize(mean = rowMeans(across(starts_with("year")), na.rm = TRUE)) %>%
  head(n = 5)
```

```
# A tibble: 5 × 2
  country         mean
  <chr>          <dbl>
1 Afghanistan      NaN
2 Albania          NaN
3 Algeria          NaN
4 American Samoa   NaN
5 Andorra          NaN
```

`colMeans()` example

Get means for each column.

Let's see what the mean is across each column (year):

```
tb %>%
  select(starts_with("year")) %>%
  colMeans(na.rm = TRUE) %>%
  head(n = 5)
```

```
numeric(0)
```

```
tb %>%
  summarize(across(starts_with("year"), ~mean(.x, na.rm = TRUE)))
```

```
# A tibble: 1 × 0
```

* New! * Many dplyr functions now have a `.by=` argument

Pipe `CO_heat` into `group_by`, then pipe that into `summarize`:

```
CO_heat %>%
  group_by(COUNTY) %>%
  summarize(avg_rate = mean(RATE, na.rm = TRUE),
            max_rate = max(RATE, na.rm = TRUE))
```

is the same as..

```
CO_heat %>%
  summarize(avg_rate = mean(RATE, na.rm = TRUE),
            max_rate = max(RATE, na.rm = TRUE),
            .by = COUNTY)
```